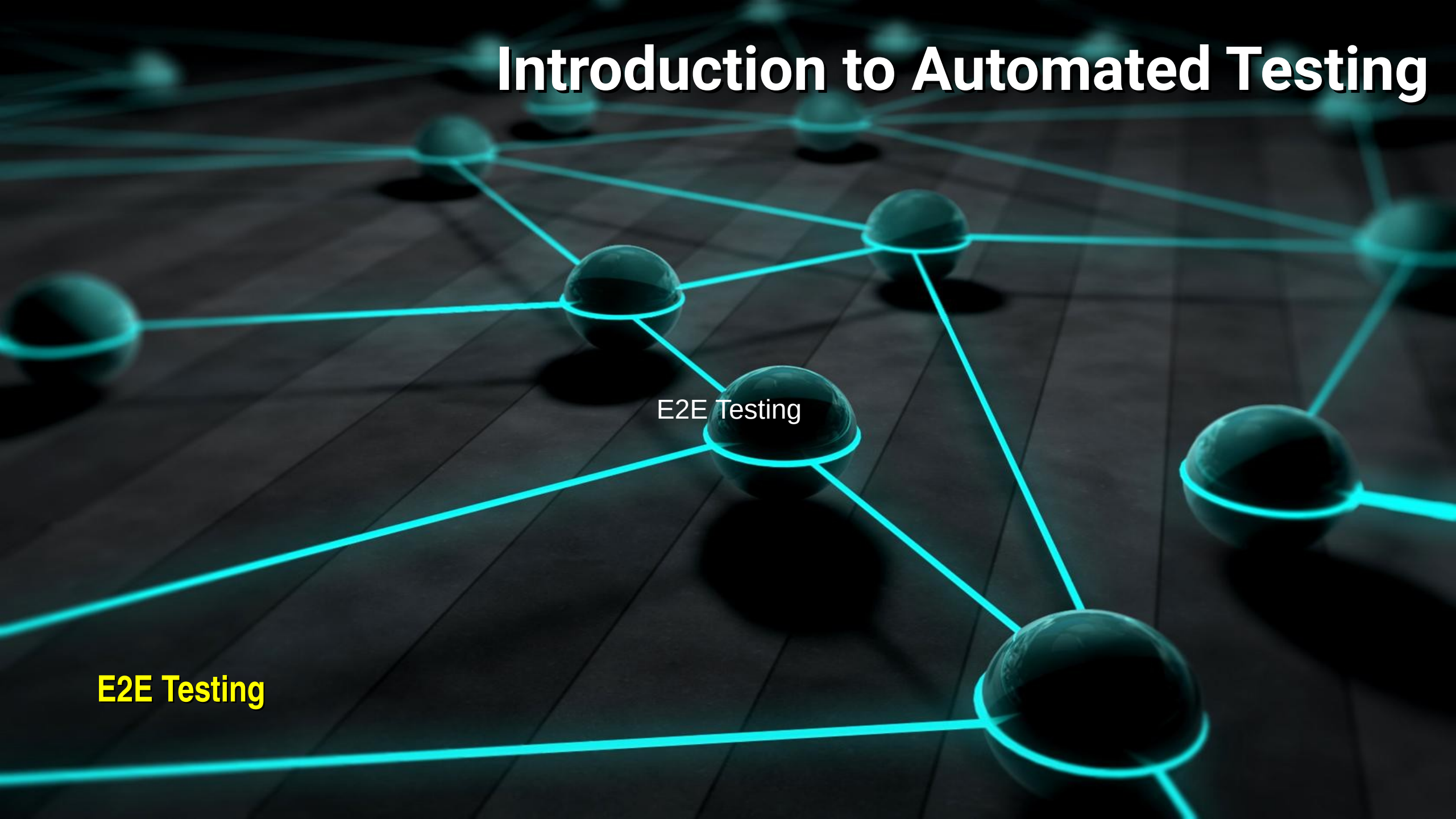


Introduction to Automated Testing

A network diagram featuring several glowing blue spheres connected by thin, bright blue lines. The spheres are arranged in a non-uniform pattern across the frame. The background is dark with a subtle grid of light blue lines. The overall aesthetic is high-tech and digital.

E2E Testing

E2E Testing

E2E Testing

- Definition
 - End-to-end testing validates a complete user workflow through the fully integrated, deployed system.
 - Enters where a user enters
 - Browser, mobile app, CLI
 - Traverses every real layer
 - UI, API, services, database, third parties
 - Verifies what the user can observe
- The question it answers:
 - "Can a real person actually accomplish this?"
- E2E gives the most confidence per test
 - And costs the most per test to write, run and maintain.

Level	Scope	Speed	Confidence per test
Unit	One function	~5 ms	Low
Integration	Components together	10 ms - 10 s	Medium
E2E	Whole deployed system	10 s - minutes	Highest



What Only E2E Can Catch

- Defects that are structurally invisible below this level:
 - Deployment and configuration errors in the real environment
 - Routing, proxy, load balancer and DNS problems
 - Authentication flows across redirects and domains
 - CORS and Content Security Policy failures
 - Session and cookie handling
 - Browser-specific JavaScript and rendering behaviour
 - The moment where all your correct components meet and fail anyway
- Everything worked in isolation
 - E2E asks whether it works assembled.



What to Automate

- Automate critical user journeys only.
 - The test:
 - "If this broke in production, would we find out from customers within the hour?"
 - Typical journeys worth E2E coverage:
 - Register / log in
 - Search then add to basket then checkout then pay
 - Submit the primary business transaction
 - Password reset
 - Ten to thirty tests is a healthy E2E suite for most products.

Don't test at E2E	Test it here instead
Validation rule permutations	Unit
Business calculations	Unit
API contract shapes	Integration / contract
Error message wording	Unit or component
Every field combination	Unit, data-driven
Database query correctness	Integration



Selenium Architecture

- Selenium is a set of integrated tools:
 - WebDriver
 - The API and W3C standard for browser automation
 - Language bindings
 - Java, Python, C#, Ruby, JavaScript, Kotlin
 - Drivers
 - chromedriver, geckodriver, msedgedriver
 - Grid
 - Distributes tests across machines and browsers
 - Selenium Manager
 - Resolves and downloads drivers automatically
 - WebDriver became a W3C standard in 2018
 - This is why Selenium works consistently across browser vendors.

```
Your test code (Java / Python)
    |
    | Selenium client binding
    v
HTTP request (W3C WebDriver protocol, JSON over HTTP)
    |
    v
Driver executable (chromedriver / geckodriver)
    |
    | browser-native automation interface
    v
Browser → renders, clicks, returns result
```



Selenium Manager

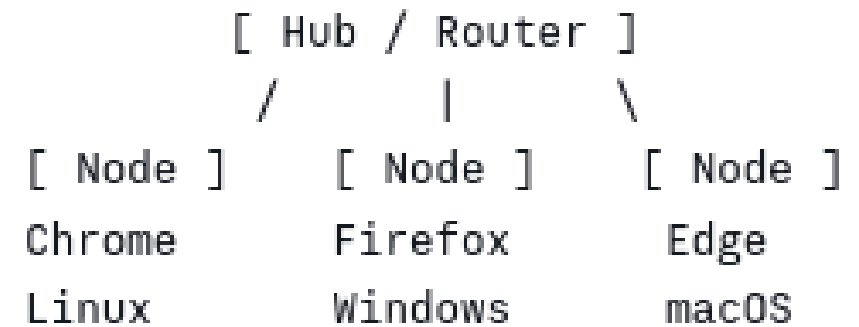
- Before 4.6
 - Everyone hand-managed driver binaries and version mismatches.
 - Now the code on the right just works
 - Selenium Manager detects the browser version, downloads the matching driver, and caches it.

```
WebDriver driver = new ChromeDriver(); // driver resolved automatically
```



Selenium Grid

- Run tests in parallel across many machines
 - Cross-browser and cross-platform coverage
 - Same test code, different RemoteWebDriver target
- Commercial equivalents
 - BrowserStack, Sauce Labs, LambdaTest



The Locator Strategies

- The WebDriver API needs to find the elements to test in the DOM tree
 - It bypasses the screen rendering
 - To find the element, it uses a locator
 - Based on how a browser finds elements in a DOM tree
 - For example, how the CSS engine finds elements to apply styling to

Strategy	Example
id	<code>By.id("submit")</code>
name	<code>By.name("email")</code>
className	<code>By.className("btn")</code>
tagName	<code>By.tagName("input")</code>
linkText	<code>By.linkText("Sign out")</code>
partialLinkText	<code>By.partialLinkText("Sign")</code>
cssSelector	<code>By.cssSelector("[data-testid='submit']")</code>
xpath	<code>By.xpath("//button[@type='submit']")</code>



Locator Priority

- Best to worst, in practice:
 - Dedicated test attribute
 - [data-testid='submit-order']
 - Id: if it is stable and meaningful
 - Name: for form fields
 - Semantic CSS
 - button[type='submit']
 - Text-based
 - linkText, or XPath on visible text
 - Structural CSS
 - .card > div:nth-child(3)
 - Positional XPath
 - /html/body/div[2]/div/form/div[3]/input
- The last two strategies break every time the layout changes.

Strategy	Example
id	By.id("submit")
name	By.name("email")
className	By.className("btn")
tagName	By.tagName("input")
linkText	By.linkText("Sign out")
partialLinkText	By.partialLinkText("Sign")
cssSelector	By.cssSelector("[data-testid='submit']")
xpath	By.xpath("//button[@type='submit']")



Brittle vs Resilient

- In the example
 - The second locator survives a CSS framework migration
 - The first does not survive adding a `<div>` element
- Best solution – test ids
 - Five-minute change for a developer
 - Eliminates an entire category of test maintenance
 - Engineering principle: include facilities for testing in the design

```
# BRITTLE – coupled to layout and styling
driver.find_element(By.XPATH, "/html/body/div[2]/div/form/div[3]/input")
driver.find_element(By.CSS_SELECTOR, ".btn-primary.mt-4.rounded-lg")

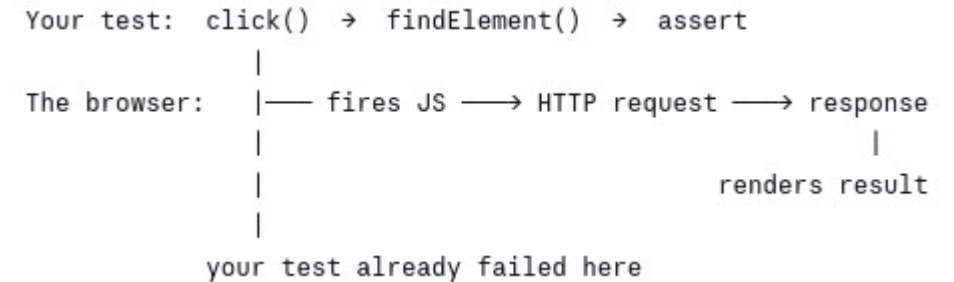
# RESILIENT – coupled to intent
driver.find_element(By.CSS_SELECTOR, "[data-testid='submit-order']")
```

```
<button data-testid="submit-order" class="btn btn-primary">
  Place Order
</button>
```



Synchronization

- Modern web applications are asynchronous.
 - The browser does not tell your test when it has finished
- Do not pause or wait directly in code
 - Example on the right will produce flaky results
 - Why it is always wrong:
 - Too short on a slow day results in a flaky failure
 - Too long on a fast day means wasted seconds
 - 200 tests × 3 s of unnecessary sleep = 10 minutes of nothing



```
driver.findElement(By.id("submit")).click();
Thread.sleep(3000); // ← never
assertEquals("Success", driver.findElement(By.id("msg")).getText());
```



Explicit Waits

- Explicit waits
 - Returns as soon as the condition is true
 - Fails at the deadline if it never becomes true
 - Fast when the app is fast.
 - Patient when it is slow.

```
java
// Java
new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.textToBe(By.id("msg"), "Success"));
```

```
python
# Python
WebDriverWait(driver, 10).until(
    EC.text_to_be_present_in_element((By.ID, "msg"), "Success"))
```

Useful Expected Conditions

Condition	Use when
<code>presenceOfElementLocated</code>	Element exists in the DOM
<code>visibilityOfElementLocated</code>	Element exists and is visible
<code>elementToBeClickable</code>	Visible and enabled
<code>textToBe</code>	Content has updated
<code>invisibilityOfElementLocated</code>	Spinner has disappeared
<code>urlContains</code>	Navigation completed
<code>numberOfElementsToBe</code>	List has finished loading



Design Patterns

- Without a pattern
 - If the code on the right were part of 40 tests
 - Then the login page changes
 - Forty changes to the test code has to be made

```
@Test void userCanLogIn() {  
    driver.get("https://app.example.com/login");  
    driver.findElement(By.id("username")).sendKeys("ann@example.com");  
    driver.findElement(By.id("password")).sendKeys("secret");  
    driver.findElement(By.id("login-btn")).click();  
    assertTrue(driver.findElement(By.id("welcome")).isDisplayed());  
}
```



Design Patterns

- The Page Object Model
 - Encapsulate each page's locators and interactions in a class.
 - Locators live in one place
 - Tests read as business intent, not DOM manipulation
 - A page change is a one-file fix
 - Methods return the next page object, modeling navigation

```
public class LoginPage {
    private final WebDriver driver;

    private final By username = By.cssSelector("[data-testid='username']");
    private final By password = By.cssSelector("[data-testid='password']");
    private final By submit = By.cssSelector("[data-testid='login']");

    public LoginPage(WebDriver driver) { this.driver = driver; }

    public DashboardPage logInAs(String user, String pass) {
        driver.findElement(username).sendKeys(user);
        driver.findElement(password).sendKeys(pass);
        driver.findElement(submit).click();
        return new DashboardPage(driver);
    }
}
```

```
@Test void userCanLogIn() {
    DashboardPage dashboard = new LoginPage(driver)
        .logInAs("ann@example.com", "secret");

    assertTrue(dashboard.isWelcomeMessageVisible());
}
```



Design Patterns

- Page Object rules

- Do

- Return the page object representing the resulting page
 - Expose behaviour (loginAs) not widgets (getUsernameField)
 - Locate elements freshly inside each method

- Don't

- Put assertions in page objects
 - Expose WebElement to the test
 - Model a whole page if a component is the right unit

```
public class LoginPage {
    private final WebDriver driver;

    private final By username = By.cssSelector("[data-testid='username']");
    private final By password = By.cssSelector("[data-testid='password']");
    private final By submit = By.cssSelector("[data-testid='login']");

    public LoginPage(WebDriver driver) { this.driver = driver; }

    public DashboardPage loginAs(String user, String pass) {
        driver.findElement(username).sendKeys(user);
        driver.findElement(password).sendKeys(pass);
        driver.findElement(submit).click();
        return new DashboardPage(driver);
    }
}
```

```
@Test void userCanLogIn() {
    DashboardPage dashboard = new LoginPage(driver)
        .loginAs("ann@example.com", "secret");

    assertTrue(dashboard.isWelcomeMessageVisible());
}
```



Design Patterns

- The assertions debate
 - Purist view
 - Page objects model the page; tests own assertions.
 - Pragmatic view
 - A page object may expose query methods
 - Like `isErrorDisplayed()`
 - Tests assert on query methods
 - Tests may assert on invariants like the page actually loaded.
 - A workable strategy
 - Page objects answer questions about boolean, String, List data
 - Tests make claims like `assertTrue(...)`
 - Exception: a Page may verify it is the page it claims to be

```
public class LoginPage {
    private final WebDriver driver;

    private final By username = By.cssSelector("[data-testid='username']");
    private final By password = By.cssSelector("[data-testid='password']");
    private final By submit = By.cssSelector("[data-testid='login']");

    public LoginPage(WebDriver driver) { this.driver = driver; }

    public DashboardPage logInAs(String user, String pass) {
        driver.findElement(username).sendKeys(user);
        driver.findElement(password).sendKeys(pass);
        driver.findElement(submit).click();
        return new DashboardPage(driver);
    }
}
```

```
@Test void userCanLogIn() {
    DashboardPage dashboard = new LoginPage(driver)
        .logInAs("ann@example.com", "secret");

    assertTrue(dashboard.isWelcomeMessageVisible());
}
```



E2E in CI/CD

- E2E requires a deployed system
 - It belongs after deployment to a test environment
 - Never in the commit stage.
 - Do not run every test on every browser
 - *Run the smoke tier across the browser matrix*
 - *Run the rest on one browser.*

COMMIT STAGE	ACCEPTANCE STAGE	POST-DEPLOY
< 10 minutes	30-60 minutes	continuous
-----	-----	-----
Unit tests	Broad integration	Smoke suite
Narrow integration	►► E2E JOURNEYS ◄◄	Synthetic monitoring
Static analysis	Performance	Canary checks

Tier	Size	When	Budget
Smoke	5-10 tests	Every deploy	< 3 min
Core journeys	20-40 tests	Every pipeline run	< 15 min
Full regression	100+	Nightly	hours
Cross-browser	Smoke only	Nightly / pre-release	—



E2E in CI/CD

- Headless execution
 - Bypasses any screen rendering code in the browser
 - Required on CI agents with no display
 - Faster and less resource-hungry
 - Set the window size explicitly
 - *The default headless viewport differs from a real browser window*
 - *Responsive layouts will hide elements your local headed run could see.*
 - Run headed locally, headless in CI
 - *Expect some divergence that needs to be accounted for*

```
ChromeOptions options = new ChromeOptions();  
options.addArguments("--headless=new");  
options.addArguments("--window-size=1920,1080");
```



E2E in CI/CD

- Parallelization
 - The only realistic way to keep E2E runtimes acceptable.
 - *Selenium Grid, or a cloud provider*
 - *JUnit 5: `junit.jupiter.execution.parallel.enabled=true`*
 - *pytest: `pytest -n 4` via `pytest-xdist`*
 - Preconditions
 - *Parallelism is unforgiving:*
 - *One WebDriver instance per thread, never shared*
 - *No shared mutable test data*
 - *Unique accounts and records per test*
 - No dependence on execution order



E2E in CI/CD

- Capture diagnostics on failure
 - An E2E failure in CI must be diagnosable without a local reproduction.
 - Capture on every failure:
 - *Screenshot at the point of failure*
 - *Page source (HTML at failure)*
 - *Browser console logs*
 - *Network log (HAR)*
 - *Video of the run, if your provider supports it*



Playwright

- Microsoft, released 2020
 - The main modern alternative to Selenium.
 - Architectural differences:
 - *Communicates over a persistent WebSocket, not per-command HTTP*
 - *Browser contexts rather than browser processes*
 - *Bundled browsers, pinned to the Playwright version*
 - *Auto-waiting built into every action*

	Selenium	Playwright
Standard	W3C WebDriver	Proprietary protocol
Auto-waiting	Manual	Built in
Language bindings	Java, Py, C#, Rb, JS, Kt	JS/TS, Py, Java, .NET
Browser install	Separate	Bundled
Network interception	Limited (CDP)	First class
Trace / debug tooling	Basic	Trace Viewer
Test isolation cost	New browser (slow)	Context (fast)
Ecosystem and jobs	Very large	Growing fast
Legacy browser support	Broader	Modern only



Playwright

- Choose Selenium when
 - You need a W3C-standard, vendor-neutral tool
 - You need broad language support or existing Grid infrastructure
 - You must support older or unusual browsers
 - Your organization already has Selenium expertise
- Choose Playwright when
 - Starting fresh on a modern web application
 - Synchronization flake is your dominant cost
 - You want network interception and tracing built in
 - Fast parallel isolation matters

	Selenium	Playwright
Standard	W3C WebDriver	Proprietary protocol
Auto-waiting	Manual	Built in
Language bindings	Java, Py, C#, Rb, JS, Kt	JS/TS, Py, Java, .NET
Browser install	Separate	Bundled
Network interception	Limited (CDP)	First class
Trace / debug tooling	Basic	Trace Viewer
Test isolation cost	New browser (slow)	Context (fast)
Ecosystem and jobs	Very large	Growing fast
Legacy browser support	Broader	Modern only



Common Mistakes

- Test interdependence
 - Consequences:
 - *Cannot run one test alone*
 - *Cannot run in parallel*
 - *One failure cascades into three*
 - *Reordering breaks everything*
 - Every test should create its own state and clean up after itself.

```
@Test void a_createsAccount() { /* creates "testuser" */ }  
@Test void b_logsIn()         { /* logs in as "testuser" */ }  
@Test void c_deletesAccount() { /* deletes "testuser" */ }
```



Common Mistakes

- Asserting on volatile content
 - Assert on the contract with the user
 - Do not assert on incidental rendering detail
 - Common cause: making assertions too specific or detailed to satisfy test requirements

```
// FRAGILE
assertEquals("Order #48291 placed on 12 Aug 2026 at 14:33",
            confirmation.getText());

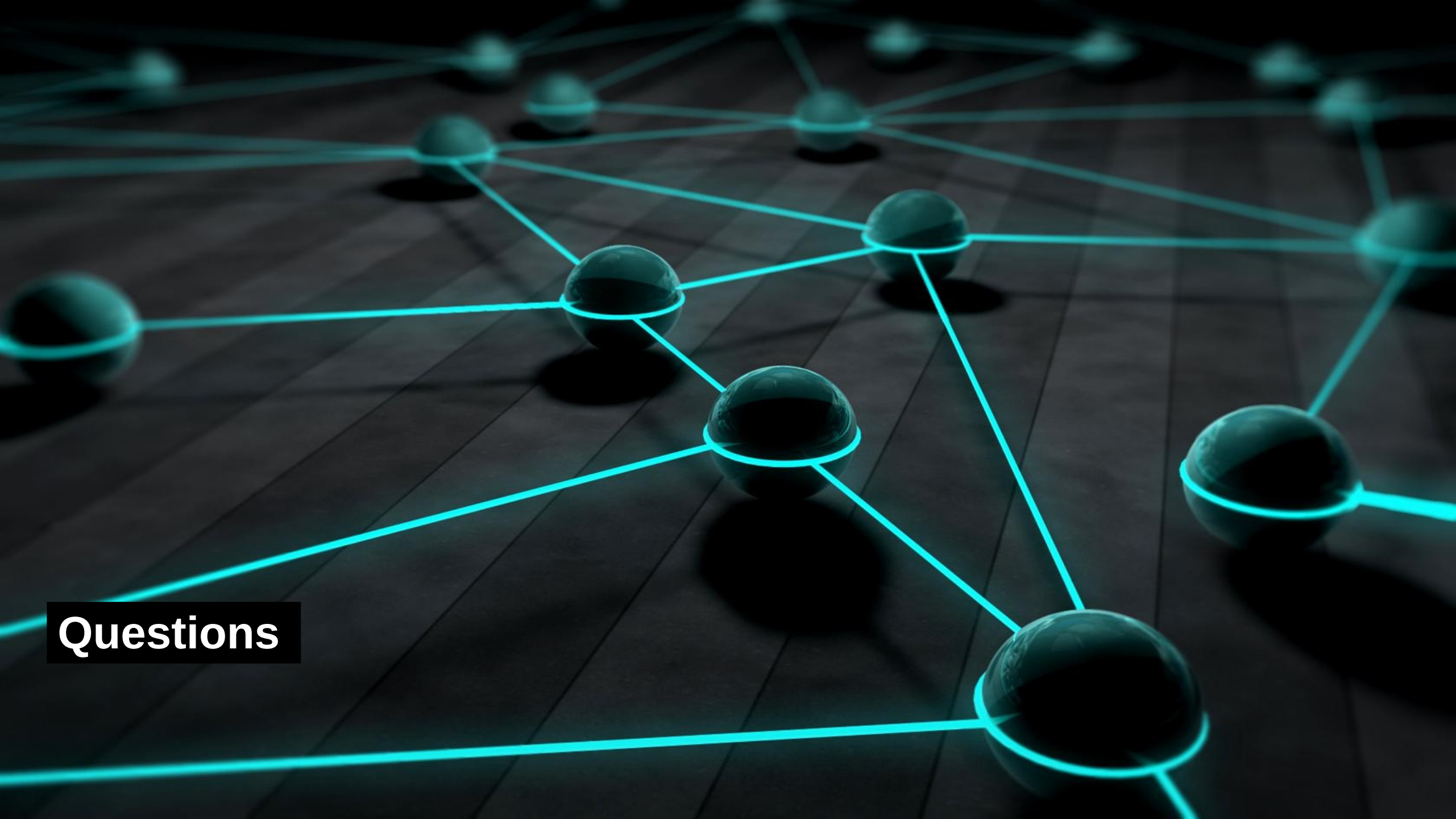
// ROBUST
assertTrue(confirmation.getText().contains("Order placed"));
assertNotNull(confirmation.getOrderNumber());
```



Common Mistakes

- No environment parity
 - Passes locally but fails in CI every time.
 - Usual causes:
 - *Headed locally, headless in CI, different viewport*
 - *Different browser version*
 - *Faster local machine hides race conditions*
 - *Local test data that does not exist in CI*
 - *Timezone or locale differences*
 - Run headless locally at least once before pushing.





Questions